

Field Asset Health Monitor

Predictive Maintenance on Industrial Sensor Data

Figure Cover



MetroPT-3 air production unit in service — the metro train whose compressor this pipeline monitors.

A time-series anomaly-detection pipeline forecasting
air-compressor failures before they occur

MetroPT-3 Metro Train Air Production Unit

1.5M sensor readings · 7 months · 4 documented failures

1 Objective

Industrial compressors fail. The question this project answers is not *did it fail* — that is trivial after the fact — but **can we warn an engineer hours before it fails**, from noisy sensor data, reliably enough to schedule maintenance instead of suffering a breakdown.

The dataset is the MetroPT-3 air production unit of a metro train: 1,516,948 readings across seven months (February–September 2020), seven analog and eight digital signals, and four documented air-leak failures. The deliverable is an honest, per-failure answer to *how much warning each failure gives* — and a runnable pipeline that produces it.

Why this is the hard version

Predicting a failure *before* it manifests is fundamentally different from detecting one while it happens. This project is built around four disciplines that separate honest prediction from after-the-fact description:

- **Train on healthy data only** — four failures cannot supervise a classifier; failures are evaluation targets, never training data.
- **Time-ordered splits**, never random — adjacent windows are near-duplicates, so shuffling would leak the answer.
- **Label the lead-up, not the failure window** — the model learns to warn, not to describe an already-broken machine.
- **Evaluate per failure** — the four failures have distinct, multidirectional signatures.

2 Data & Methodology

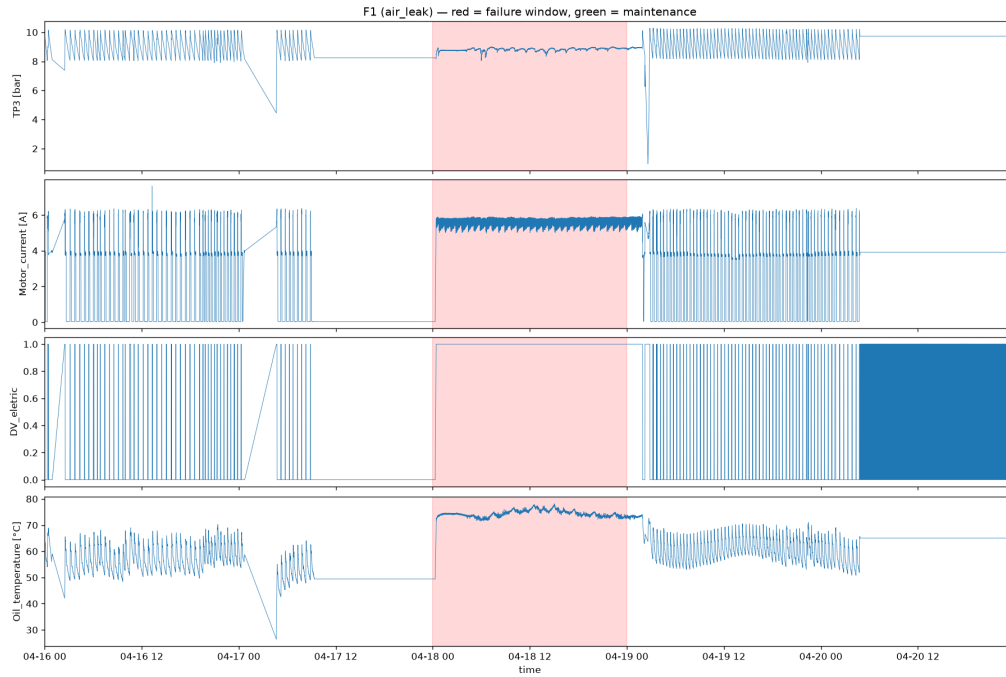
The pipeline runs in five stages, each reading and writing a data artifact so the whole run is reproducible from a single command.

Stage	In → Out	What it does
Preprocess	raw CSV → parquet	typed load, 6 integrity checks, failure windows
EDA	parquet	distributions, correlations, state-mixture analysis
Anomaly context	parquet	per-failure portraits, 6-category labeling
Features	→ features.parquet	gap-aware 1h grid, 6 feature families, scaling
Modeling	→ scores + report	anomaly detection, per-failure evaluation

Preprocessing & integrity

A typed load cut memory from 224 MB to 104 MB. Six integrity checks — which *raise* on bad data, halting the pipeline — verify row counts, timestamp monotonicity, value ranges, and the failure-window sources. A gap inventory found 331 recording gaps totalling ~18% missing time, a fact that shapes every downstream windowing decision.

Figure 2.1

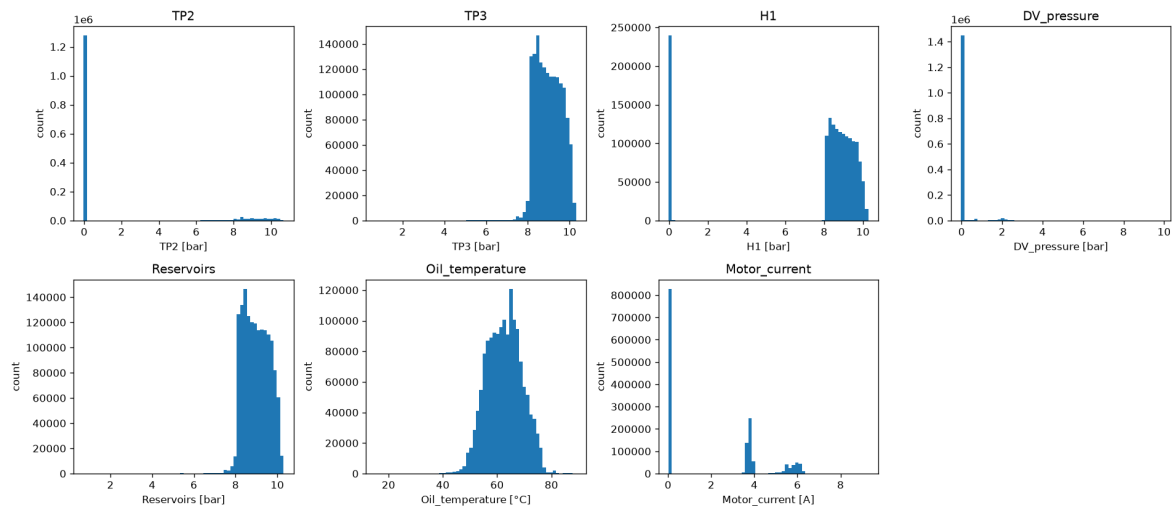


Failure portrait (F1): the sensor context around a documented failure — TP3, motor current, DV_electric, oil temperature. Red = failure window, green = maintenance.

Exploratory data analysis

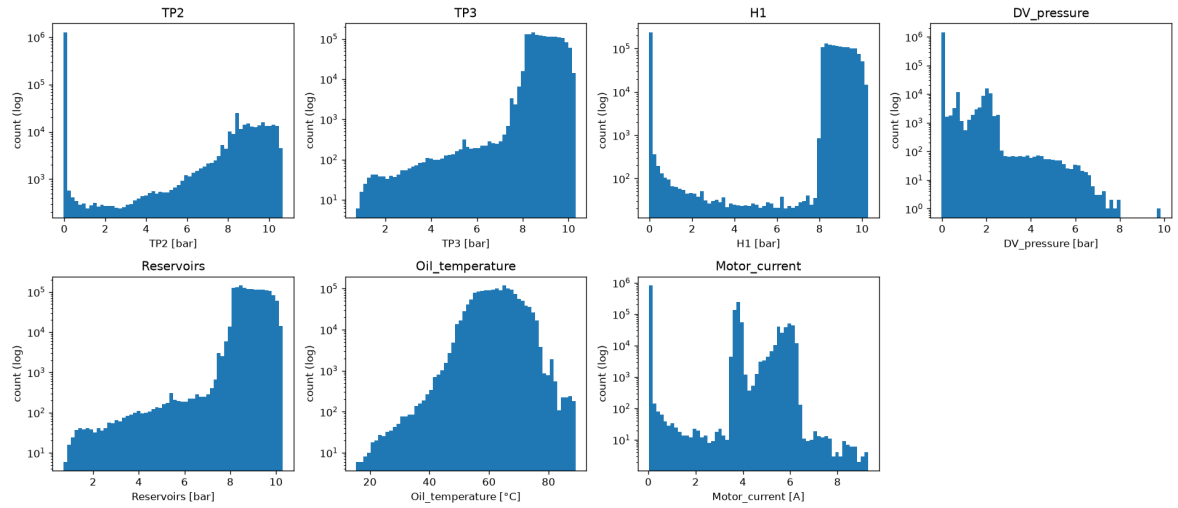
The seven analog signals are strongly bimodal — a large idle mass at zero plus an active-operation lobe — so distributions are shown on both linear and log count scales to make the sparse active tail legible.

Figure 2.2



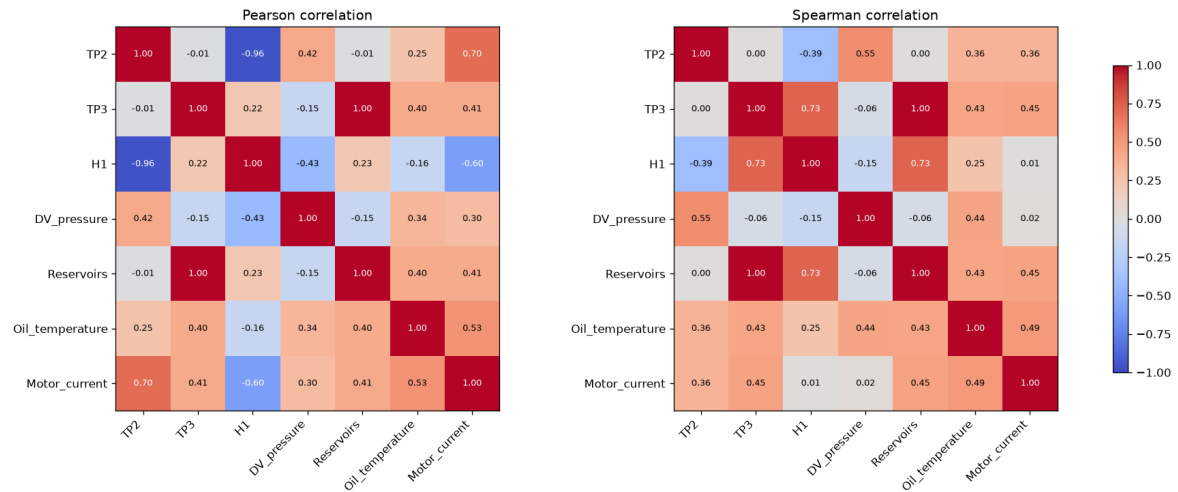
Analog signal distributions (linear count scale): TP2, TP3, H1, DV_pressure, Reservoirs, Oil_temperature, Motor_current.

Figure 2.3



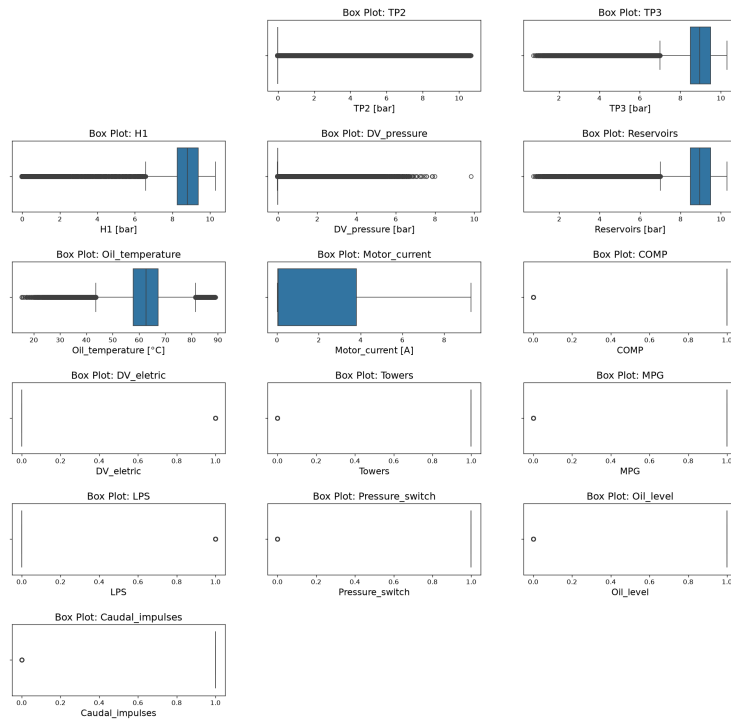
The same seven analog distributions on a log count scale — the active lobe and rare high-pressure events the linear view hides.

Figure 2.4



Pearson and Spearman correlation across the analog signals — TP3 and Reservoirs move together; TP2 and H1 are strongly anti-correlated.

Figure 2.5



Per-signal box plots across all signals — the analog tails and the binary digital channels that inform the labeling scheme.

Labeling — the payoff of investigation

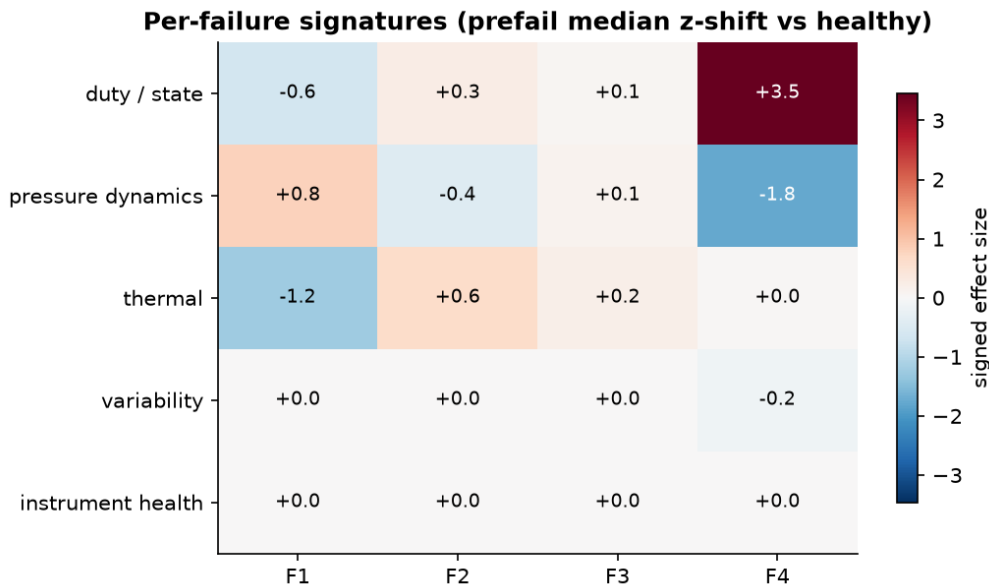
A naive healthy/failure split would poison the healthy baseline. Instead, a six-category scheme was built, two of whose categories are *findings*, not givens: a **12-day undocumented degraded span** (a leak running between a failure and its repair) and a **~20-hour frozen-instrument fault** (sensors stuck, machine fine), each carved out by investigation so that windows labeled healthy are genuinely healthy.

3 Feature Engineering

Sensor readings were aggregated into a gap-aware grid of 4,060 one-hour windows (no window bridges a recording gap). Six feature families were built and each validated by whether it separates failures — families that did not earn their place (calendar, spectral) were analysed and excluded.

- **Duty & state-occupancy** — load fraction and time-in-each-motor-mode.
- **Pressure dynamics** — a per-idle-stretch pressure-decay slope: the *direct* air-leak meter (reservoir pressure falling while the machine rests).
- **Thermal** — oil level, trend, and an **oil residual** against a healthy oil-vs-workload baseline, separating ‘hot because busy’ from ‘hot because failing’.
- **Variability** — cycle-duration coefficient of variation and trend: the signature that rescued the hardest failure.
- **Instrument-health** — antiphase and frozen-sensor flags: a trust axis orthogonal to machine health.

Figure 3.1



Per-failure effect-size heatmap (prefail median z-shift vs healthy): each feature family vs each failure — the distinct, multidirectional signatures (F4 on duty/state, F1 on thermal, opposite pressure signs).

A recurring lesson

Features were assigned to failures by *measurement*, not expectation. The pressure-decay slope, first computed window-wide, averaged a sawtooth to zero and separated nothing; refitting it per idle-stretch recovered the leak signal. A 'lock-up' feature built to catch one failure caught a different one. Informative missingness (idle/locked machine states) was modeled explicitly rather than dropped.

4 Modeling & Results

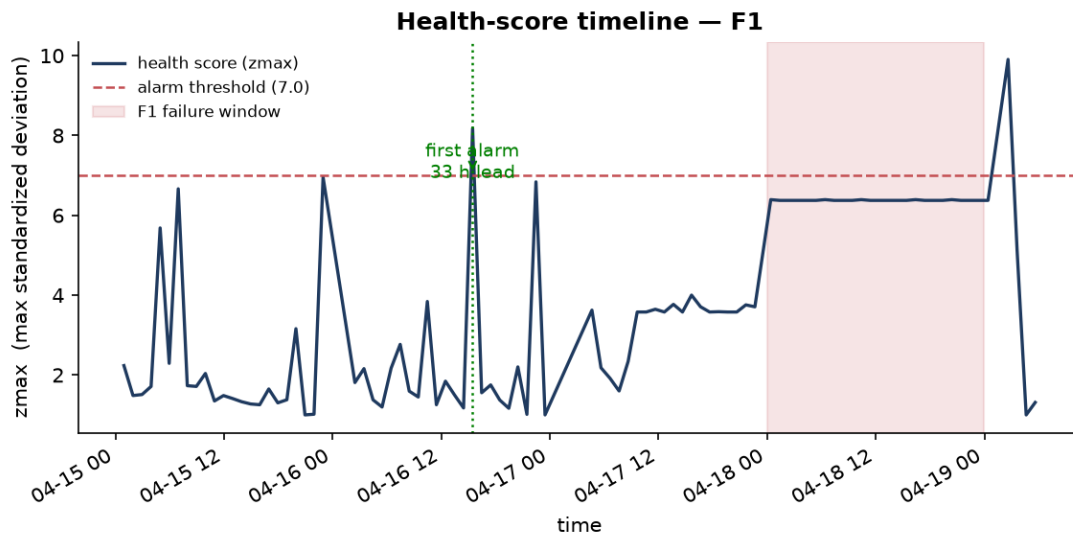
Three paradigms were compared under one honest protocol — anomaly detection, supervised leave-one-failure-out, and forecast-residual. The winner is the simplest: **zmax**, the maximum standardized deviation of any feature from healthy operation. It needs no training, is fully interpretable, and beat IsolationForest, Mahalanobis, seven supervised classifiers, and two forecast-residual variants on every target.

Per-failure detection (the honest scorecard)

Failure	ROC-AUC	PR-AUC	Lead time	Verdict
F1 (idle-then-step)	0.86	0.30	32.5 h	strong
F4 (gradual leak)	0.80	0.25	8 h	good
F3 (erratic cycling)	0.70	0.40	9.6 h	hardest — rescued
F2 (short leak)	0.64	0.07	—	weak, no early warning
Degraded span	0.58	0.19	—	trend only

“These are credible predictive-maintenance numbers — real early warning with a real precision / recall / false-alarm tradeoff — not the leakage-driven ~100% accuracy a window-labeled, randomly-split setup produces.”

Figure 4.1



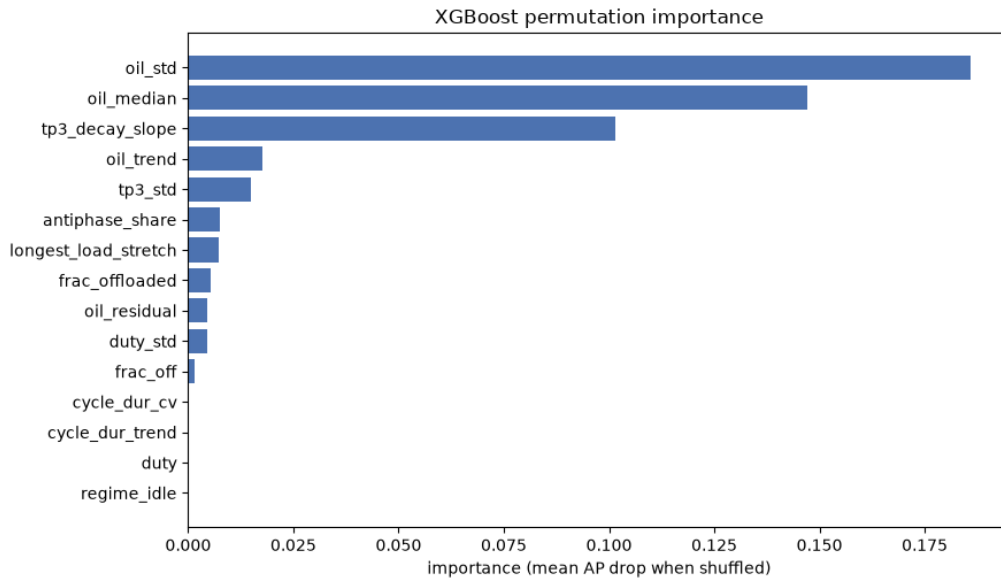
Health-score timeline for F1: the zmax score climbing across the alarm threshold (7.0) — first alarm ~33 h before the failure window (shaded red).

The two findings that matter most

F3 — the hardest case, rescued. F3 was invisible to every level, rate, and even spectral (Lomb-Scargle) feature; it looked genuinely undetectable. It was recovered by the **cycle-dynamics features** (its short-cycling signature) and confirmed by the leave-one-failure-out result that F3's pattern — unlike F4's — is learnable from the other failures.

A failure taxonomy. Leave-one-failure-out training revealed that anomaly detection and supervised learning catch *opposite* failures: F4 is the easiest to flag as an anomaly (it is unique, so it deviates most from healthy) but the *hardest* to learn from other failures (unique → unseen); F3 is the reverse. A mature system uses both.

Figure 4.2



Feature importance (prefail-only), XGBoost permutation importance: oil dispersion (*oil_std*), oil level (*oil_median*), and the pressure-decay slope (*tp3_decay_slope*) as the verified predictive signals.

What drives the alarm — verified predictive, not descriptive

Importance was computed on the lead-up only, excluding during-failure windows (which would let the model describe the failure rather than predict it). Two physically-grounded signals dominate: the **pressure-decay slope** (the air leak measured directly) and **oil temperature** (the same leak measured indirectly — the compressor overworks to compensate and heats up). Two sensors, one root cause — cross-confirmed by independent importance methods.

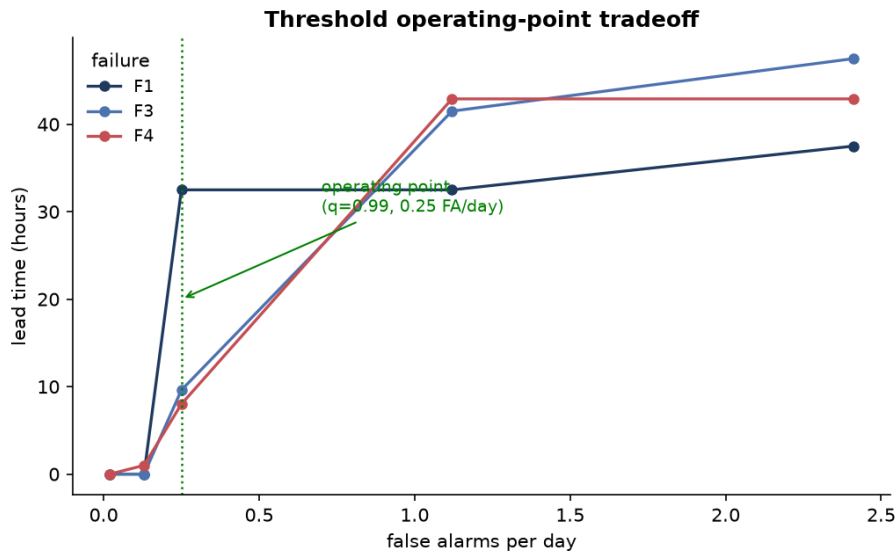
5 From Analysis to Alarm

The pipeline's scoring stage emits, per window, a health score, an alert flag, and the **driving feature** — so an alert is a work order, not a black box:

■ 2020-07-08 18:00 — health score 23.9 (threshold 7.0), driver: *tp3_decay_slope*
→ suspected air leak, ~8 h to likely failure.

The alarm threshold is not a fixed choice but a tunable operating point: loosening it buys more lead time at the cost of more false alarms, a tradeoff presented as a menu for the engineer's failure-cost context.

Figure 5.1



Threshold operating-point curve: lead time vs false alarms per day across threshold quantiles, per failure — the operating point ($q=0.99$, 0.25 FA/day) marked.

One command, end to end

```
poetry run python run_pipeline.py --config configs/config.yaml --stage all
```

Raw CSV in, per-failure evaluation and scored alerts out. The runner reproduces the notebook results exactly, verifying that the `src/fahm` package — not the notebooks — is the single source of truth.

6 Honest Limitations

- **F2 detects but does not warn early** — a mild thermal precursor gives moderate separability but no reliable lead time.
- **Slow degradation is weak** (0.58) — better read as a trend than a binary alarm.
- **Low recall** at the conservative threshold — the system favors trustworthy early warning over exhaustive flagging; one early true positive suffices to warn.

Reporting these plainly is the point. A documented capability envelope is more useful to a maintenance engineer — and more credible to a reviewer — than a suspiciously perfect score.